How to Start a Project with an LLM — Step by Step (v1.2.0)

Version: `1.2.0`
Date: 2026-10-18
Filename: `START-A-PROJECT-STEP-BY-STEP-v1.2.0.md`
Purpose: A repeatable, beginner-friendly procedure for starting a **small or large software project** that you will develop *with* a coding agent such as Codex, OpenCode, or Claude Code.

> **What this guide does:** It gets you from an idea to a real, version-controlled project folder (the **scaffold**) with useful instructions, data contracts where needed, approved skills, and approved MCP servers. **It does not freeze the folder layout forever.** The project and its documentation will evolve as you and your agent build it.

Before you begin

- **You** decide what you want and approve decisions that matter. **The LLM** helps plan and write files. **The coding agent** works in the actual project folder and can help build and test software.
- An **LLM chat** can help draft plans even before you create a project folder. A **coding agent** (Codex, OpenCode, Claude Code, etc.) typically operates on files in that folder.
- **Skill:** a reusable set of instructions or procedures for an agent. **MCP server:** a connection that exposes tools or data to an agent. They are not interchangeable.
- **Scaffold:** the initial working directory, files, configuration and tests—not the finished application.
- You need a terminal, Git, and a coding agent. Some suggested skills use Node.js/npm (`npx`). Install only the tools your selected

stack needs. Commands below assume a Bash terminal (Linux/macOS/Git Bash/WSL); ask your agent for PowerShell/CMD equivalents if necessary.

- **Follow the steps in order.** When a step says *if applicable*, explicitly record **Not applicable** and continue. You should not have to install an unnecessary skill, MCP server, database, UI framework, or validation library for a tiny project.
- **Never paste passwords, tokens, credentials, confidential production data, or personal data into prompts, documentation or Git.** Use synthetic or sanitized examples.

Choose the smallest setup that fits your project

Do not create files just to fill a template. The steps below apply to all projects, but the amount of documentation should match the work. When a step is optional, write **Not applicable** in your planning notes and move on.

Project type	Begin with	Add when needed
Tiny script or experiment	`README.md`, `AGENTS.md`, a runnable program, Git	Brief `PROJECT_SPEC.md`; `ARCHITECTURE.md` if there are important rules
Application with a user interface	Above, plus `DESIGN.md`	Screens, component rules, accessibility checks
Data/import/API/LLM-output project	Above, plus `DATA_CONTRACTS.md`, appropriate `schemas/` or equivalent contracts, validation tests	Migrations and compatibility checks
Complex or collaborative application	Above, plus `ARCHITECTURE.md`, appropriate skills and integrations	Expanded tests, CI, decision records, reviews

How to use this guide: Work down the numbered steps. *You* approve decisions; the *planning chat* drafts specifications; your *coding agent* creates and changes files in the actual repository. A copy/paste prompt can be adapted to your situation. Every step identifies its expected result. You do not need to complete a large paperwork exercise to scaffold a small script.

Which project file does what?

File or folder	Audience	Role	Create or update when
`README.md`	People	What this project does and how to start it	Initial scaffold and user-facing changes
`PROJECT_SPEC.md`	You and agent	Scope, acceptance criteria, project goals	Planning and changed requirements

File or folder	Audience	Role	Create or update when
`AGENTS.md`	Coding agents	Commands, code conventions, safety restrictions, where other rules live	After real scaffold exists, then as workflows change
`DESIGN.md`	Agent / UI contributors	Concrete UI design tokens and interaction rules	UI exists or changes
`ARCHITECTURE.md`	Agent / maintainers	Non-obvious invariants and rationale	Architectural decisions change
`DATA_CONTRACTS.md`	Producers, consumers, agent	Meaning and compatibility of exchanged data	Data interfaces change
`schemas/`	Programs and validators	Machine-readable JSON contracts where appropriate	A structured JSON interface exists or changes
`skills/`	Coding agent	Optional **project-local** reusable procedures, each with a `SKILL.md`	A repeated project-specific procedure is needed
`SKILLS.md`	You and agent	Inventory of selected, installed, and global reusable skills	Skill selection or version changes
`MCP_SERVERS.md`	You and agent	MCP server inventory, permissions and connection setup *without secrets*	MCP selection or configuration changes
`scripts/scaffold.sh`	You and agent	Reproducible initial setup (not a frozen project layout)	Scaffold creation or intentional template evolution

Important distinction: `README.md` explains the project to people; `AGENTS.md` explains how to work *in this repository* to a coding agent; a `SKILL.md` explains a reusable procedure that might work in many repositories. A globally installed skill need not be copied into every project's `skills/` folder. Document its name, source and invocation in `SKILLS.md` instead. Some agents use their own instruction filename or import mechanism; verify the actual supported behavior instead of assuming every agent automatically loads `AGENTS.md`.

PHASE A — Decide what you are building

STEP 1 — Explain your idea in everyday words

Write 2–5 sentences. What should the project do? Who will use it? What is the most important outcome? You do not need technical vocabulary.

You do: Write or dictate your rough idea in an LLM chat.

Result: A short, informal description of your idea.

STEP 2 — Turn the idea into a small, testable project specification

Copy and paste to the LLM:

> Here is my idea: [PASTE IDEA]. Help me create a first-version project specification. State its purpose, intended users, 3–6 core features, what is *out of scope*, how we will know the first version works, a recommended technology stack, and a simple starter folder structure. Separate facts I supplied from your assumptions. Ask me only about decisions that would materially change the plan. Save the approved specification as `PROJECT_SPEC.md`.

Read the draft, correct it, and approve it. Start with the **smallest useful version**; a large project can add features later.

Result: An approved `PROJECT_SPEC.md`.

STEP 3 — Clarify uncertain requirements (a grilling interview)

A *grilling* skill asks focused questions to expose hidden assumptions. You can conduct the interview in the planning chat **without installing a skill yet**. Later, if you install `grill-with-docs`, it can revisit and document decisions inside the repository.

Copy and paste:

> Interview me about the proposed project. Focus on missing requirements, conflicting assumptions, expected users, inputs and outputs, edge cases, success criteria, and restrictions. Ask one important question at a time. Keep a short decision log and update `PROJECT_SPEC.md` after I approve the answers. Stop when the project is clear enough to scaffold.

Result: Clearer requirements and a short list of decisions or unresolved questions. **Do not wait for every future implementation detail to be known.**

STEP 4 — Draft the three project-rule documents

These describe different concerns. **Create short, useful files rather than long speculative documents.**

Document	Question it answers	When needed
`DESIGN.md`	How should screens look and behave?	**If the project has a UI** (web, desktop, mobile, CLI with a significant interactive interface)
`ARCHITECTURE.md`	How should the system be organized, and which rules must never be broken?	**All projects**; very brief for tiny scripts
`DATA_CONTRACTS.md`	What data enters, leaves, or passes between components, and what does it mean?	**Any project with structured data exchange, files, APIs, configuration or structured LLM output**

Copy and paste:

> Using `PROJECT_SPEC.md`, draft `ARCHITECTURE.md` (system parts, important invariants and technology decisions). If there is a user interface, also draft `DESIGN.md` (design tokens, layout, components, behavior for loading/errors, responsiveness and accessibility). Identify whether the project has structured data inputs/outputs; if so, draft `DATA_CONTRACTS.md` (data producers, consumers, formats, semantics and validation rules). Keep these documents appropriate to the project's size. If something does not apply, say why. Flag contradictions among the documents before I approve them.

For a simple command-line script, `DESIGN.md` may be **Not applicable** in the planning checklist and need not exist. A tiny script may record its few technical rules directly in `AGENTS.md` and skip a separate `ARCHITECTURE.md`; for nontrivial systems, keep it explicit.

If you create `DESIGN.md`: Ask for actual values, not vague adjectives. For example, include YAML front matter or a table containing hex color tokens, font stacks, spacing scale and corner radii. Add layout, components, keyboard accessibility, loading/error states and responsive rules. This prevents one agent session from changing the design unintentionally in the next. Keep it short for a small UI.

Result: Approved rule documents appropriate to your project.

STEP 5 — Define machine-checkable data contracts (when needed)

If your project receives or produces structured data, decide **exactly what shape it has before writing the modules that consume it**. Examples: CSV imports, API requests and responses, JSON config, messages between modules, and structured LLM responses.

A **JSON Schema** describes constraints on JSON data. It can specify names, types, required fields, allowed values, numeric bounds and conditional structure. **It does not by itself prove every business rule** (such as an actuarial formula, uniqueness across database records, or equality between two separately stored totals). Use code/tests for those extra rules. For CSV, binary, database tables or other formats, document their native contract too; a JSON Schema is not automatically the right format for everything.

Copy and paste:

> From `PROJECT_SPEC.md`, `ARCHITECTURE.md` and `DATA_CONTRACTS.md`, inventory each data producer, consumer, input, output, and transformation. For JSON interfaces, create schemas in `schemas/` using an explicitly stated JSON Schema draft (prefer Draft 2020-12 unless my tools require another). Specify required fields, types, enums, nullability, units, dates, IDs, ranges, optional fields and error behavior. For non-JSON formats, document an equivalent contract. Generate **synthetic** valid, invalid and boundary examples and identify rules requiring additional code-based tests. Do not invent field names or domain semantics; list unresolved questions. Explain how contracts will be validated and versioned.

Suggested small structure (only what is needed):

```
schemas/
  inputs/
  outputs/
  config/
  examples/valid/
  examples/invalid/
tests/contracts/
```

Only create subfolders that your project actually uses. For example, an API project could use `schemas/api/create-order.schema.json`, and an agent-configuration project could use `schemas/config/opencode.schema.json`. Do not force JSON Schema onto CSV or relational data; use an appropriate format-specific contract. Establish a compatibility policy: changes that break an existing producer or consumer require explicit review, migration or coordinated release. Decide whether contract versions are carried by filenames, schema IDs, API versions or another clear mechanism.

Result: Approved `DATA_CONTRACTS.md`, relevant schema files (for example `schemas/inputs/import.schema.json`), synthetic examples and a validation plan. If there is no structured data exchange, record **Not applicable**.

PHASE B — Choose agent skills and MCP connections

Important distinction: In this phase you are **selecting and reviewing** tools. Project-local installation will happen **after the folder and Git repository exist**. This avoids a common ordering problem.

STEP 6 — Review a baseline set of reusable skills

These are **candidates to consider for nearly every project**, not an instruction to install all of them automatically. Verify their current names and sources before installing; upstream repositories may change.

Skill	Appropriate usage
`grill-me` **or** `grill-with-docs`	Challenge requirements. Use **one main interviewing approach**; `grill-with-docs` is especially suitable once project files exist and decisions should be documented.
`find-skills`	Discover skills for a newly identified project need. Discovery **does not** authorize automatic installation.
`to-prd` → `to-issues`	Turn approved requirements into a Product Requirements Document (PRD), then actionable issues. Run `to-issues` **only after** approving the PRD and preparing an issue tracker.
`improve-codebase-architecture`	Review/refactor an **existing** body of code. **Do not** run as a mandatory refactor on an empty scaffold.
`setup-matt-pocock-skills`	Configure the supporting workflow in a suitable, initialized repository, if you use that toolset.

To inspect what's actually offered (do not install blindly):

```
npx skills@latest add mattpocock/skills --list
npx skills@latest add vercel-labs/skills --list
```

Example targeted install commands (run **after Step 12**, inside the new project, if approved and supported by your coding agent):

```
npx skills@latest add mattpocock/skills --skill grill-with-docs
npx skills@latest add vercel-labs/skills --skill find-skills
npx skills@latest add mattpocock/skills --skill to-prd --skill to-issues
npx skills@latest add mattpocock/skills --skill improve-codebase-architecture
npx skills@latest add mattpocock/skills --skill setup-matt-pocock-skills
```

Optional alternative to `grill-with-docs`:

```
npx skills@latest add mattpocock/skills --skill grill-me
```

The name `prd-to-issues` may describe the desired workflow; the **two skills shown above** are `to-prd` and `to-issues`. An agent may expose skills as slash commands (such as `/to-prd`), as a named skill, or through another interface. **Ask your agent how to invoke the installed skill; do not assume all agents support identical slash commands.**

References: https://skills.sh/mattpocock/skills · https://skills.sh/vercel-labs/skills/find-skills

STEP 7 — Add YOUR required skills, then seek optional suggestions

Copy and paste:

> The skills I require are: [LIST NAMES, REPOSITORIES OR LINKS]. Keep my choices separate from optional recommendations. Check compatibility with my chosen coding agent. Recommend only additional skills that serve a distinct purpose. Explain when each should be used; do not install anything yet.

Where skills live: A skill can be installed globally in your agent environment (portable across repositories) or locally for a specific project. Do not copy a global skill into each project unless your agent

requires it or you deliberately need a pinned, project-specific version. Each local skill normally has its own `SKILL.md`, for example `skills/release-checklist/SKILL.md`. Keep a source/version/install-scope inventory in `SKILLS.md`.

Result: One proposed skill manifest: **required by user**, **recommended**, **accepted/rejected**, **reason**, **installation source**, **when to invoke**. This will become `SKILLS.md`.

STEP 8 — Audit skill conflicts before installing

Copy and paste:

> Examine documentation and configuration for every selected skill. Identify duplicates, contradictory instructions, overlapping responsibilities, dependency/version mismatches, permissions, and competing configuration files. Write `SKILL_CONFLICT_REPORT.md` with columns: Skill A, Skill B, evidence, severity (Critical/Moderate/Minor), recommended resolution, and status. Preserve user-required skills when technically and safely possible; ask me about substitutions. Separate **verified** conflicts from **potential** conflicts. If none are found, state 'No skill conflicts detected' and specify what was checked. Do not install skills with unresolved critical conflicts.

Result: An accepted `SKILL_CONFLICT_REPORT.md`. Compatible overlaps need not be removed; assign distinct responsibilities when useful.

STEP 9 — List YOUR MCP servers, then consider optional suggestions

An MCP server gives the coding agent access to capabilities such as repository operations, databases, external services, or documentation. **MCP is optional:** a simple offline script may not need any server.

Copy and paste:

> My required MCP servers are: [LIST, OR 'none']. State what each server connects to, which operations I expect, and whether the chosen coding agent supports it. Suggest only complementary servers, including their official sources and required permissions. Do not install or authorize them yet.

Result: An approved `MCP_SERVERS.md` plan, or **No MCP servers needed**.

STEP 10 — Audit MCP conflicts and permissions

Copy and paste:

> Audit every selected MCP server for duplicate tool names, competing writes, incompatible runtimes, transports, versions, ports, authentication, file/network access and excessive permissions. Write `MCP_CONFLICT_REPORT.md` with evidence, severity, resolution and verification status. Do not enable unresolved critical conflicts. Do not request secrets in chat or place credentials in repository files.

Result: An approved `MCP_CONFLICT_REPORT.md`, or a short **Not applicable** entry if there are no MCP servers. Remember that merely reviewing configuration does not prove the connection works.

PHASE C — Build the initial project folder

STEP 11 — Ask for a scaffold script and starter files

The script should **create real files** and be safe to rerun without silently destroying your edits. It should **not automatically execute unreviewed third-party installs**, connect accounts, or embed secrets. Keep installation commands in an explicit checklist or a separate reviewed setup script.

Copy and paste:

> Generate `scripts/scaffold.sh` for my approved project and selected stack. It must create a minimal, working starter project and place the actual approved contents into `PROJECT_SPEC.md`, `ARCHITECTURE.md`, `DESIGN.md` (if applicable), `DATA_CONTRACTS.md` and `schemas/` (if applicable), plus `README.md`, `SKILLS.md`, `MCP_SERVERS.md`, `SKILL_CONFLICT_REPORT.md`, and `MCP_CONFLICT_REPORT.md`. Add a suitable `.gitignore`, a minimal smoke test, and `tests/contracts/` if contracts exist. Do not create irrelevant directories. Do not overwrite edited files on rerun. Include instructions for prerequisite installation and the exact commands to run and test the starter app. Put reviewed, project-local skill and MCP setup instructions in a separate `scripts/setup-agent-tools.sh` when needed; never store secrets. Explain any agent-specific differences. Output a complete downloadable script and necessary files, not placeholders disguised as finished content.

The generated tree might look like this; **your project can be simpler or larger**. `AGENTS.md` should not be blindly generated before real files exist. Document any global skills in `SKILLS.md`; include `skills/` only if you actually maintain project-local skills:

```
my-project/
├── README.md
├── PROJECT_SPEC.md
├── ARCHITECTURE.md
├── DESIGN.md                   # UI projects only
├── DATA_CONTRACTS.md           # structured data projects only
├── AGENTS.md                   # created/reviewed with agent later
├── SKILLS.md
├── MCP_SERVERS.md
├── SKILL_CONFLICT_REPORT.md
├── MCP_CONFLICT_REPORT.md
├── schemas/                    # only when needed
├── skills/                     # only project-local skills, if needed
│   └── release-checklist/
│       └── SKILL.md
├── tests/                      # minimal smoke/contract tests
├── scripts/
│   ├── scaffold.sh
│   └── setup-agent-tools.sh    # only when needed
└── src/                        # or stack-appropriate entry point
```

Result: Downloadable scaffold script, reviewed contents and clear execution instructions.

STEP 12 — Create the folder, run the scaffold, and check the result

Save `scaffold.sh` somewhere accessible, create a folder and **run the script from that folder**. One possible Bash workflow:

```
mkdir my-project
cd my-project
mkdir -p scripts
# Copy the downloaded scaffold.sh into scripts/ here.
bash scripts/scaffold.sh
```

If the generated script expects another location or arguments, follow **its** documented invocation instead. Then list files and run the generated smoke test. Do not proceed if the scaffold did not create the expected files or a runnable starting point.

Result: A real directory containing working starter files and the approved documentation.

STEP 13 — Start Git and save the initial baseline

From the project root:

```
git init
# Check .gitignore and ensure no tokens, private data or local secrets are staged.
git status
git add .
git commit -m "chore: initial project scaffold"
```

Configure your Git identity if Git requests it. Creating a **remote** GitHub repository and pushing is a separate, optional step—ask the coding agent to help when you are ready. A local commit is already a useful recovery point.

Result: Your first version-controlled baseline. **Keep `scripts/scaffold.sh` in Git**, so you can understand how the initial folder was created. The script is a historical setup recipe, not a command to recreate or overwrite an evolving project tree. If you later extend it, make changes safe and intentional.

STEP 14 — Install approved skills and configure approved MCP servers

Now that the directory exists, use the **reviewed** commands from Steps 6–10. Install project-local tools from the project root. If you use `setup-matt-pocock-skills`, invoke it **after Git initialization**, subject to the tool's current requirements. Global installations may have been completed earlier.

Copy and paste to your coding agent:

> Follow `SKILLS.md`, `MCP_SERVERS.md` and the two conflict reports. Show me the approved installation commands and verify their sources before executing them. Install only approved project-local skills, configure only approved MCP servers, and use environment variables or a secure secrets store for credentials. Verify each skill is discoverable and each MCP server actually connects and exposes the expected tools. Record successes, failures and exact versions/revisions when available. Do not claim success if a check was not run.

Result: Verified agent tools, or an explicit list of unavailable tools and what remains to be done. **Some MCP servers may require you to authorize accounts yourself.**

STEP 15 — Open the coding agent inside the project and initialize instructions

Start Codex, OpenCode, Claude Code or another agent **with the project folder as its workspace**. Use the agent's initialization feature if one is available; some use `/init`, others use different commands or files. **Ask your agent instead of assuming `/init` is universal.**

Copy and paste:

> Inspect the files that actually exist in this repository. Create or update the agent instruction file used by this coding agent. Prefer a concise `AGENTS.md` as a shared reference where supported; create the specific file or import directive required

> by this actual agent. Do not assume `AGENTS.md` is automatically read by every tool. Reference `PROJECT_SPEC.md`, `ARCHITECTURE.md`, `DESIGN.md` (if present), `DATA_CONTRACTS.md` and `schemas/` (if present), `SKILLS.md`, `MCP_SERVERS.md`, and both conflict reports. Explain the exact build, test and validation commands; state when to use each approved skill or MCP tool. Do not copy large skill instructions into `AGENTS.md`, fabricate missing files, or include credentials. Preserve important user-written instructions.

Result: An agent that knows the repository rules and can locate the right documentation. Keep the `README.md` human-facing and put operational instructions in `AGENTS.md`, instead of duplicating them in both. **Initializing an agent is not proof that all tools work—verify discovery and connections separately.**

STEP 16 — Test the baseline and commit the agent setup

Ask the agent to run the app's smoke test, schema validation (where applicable), and any existing unit tests. Confirm that the repository instructions reference the right files. Then:

```
git status
git add .
git commit -m "chore: configure agent context and contracts"
```

If nothing changed since the last commit, Git correctly reports nothing to commit.

Result: A reproducible, tested starting point for development alongside your coding agent.

PHASE D — Build and keep the project healthy

STEP 17 — Plan the first small development task

The initial scaffold is **not** a complete application. Ask the agent to turn the approved specification into small changes with tests. If you selected Matt Pocock's workflow, use `to-prd` to refine the PRD and `to-`

`issues` **after** PRD approval and issue-tracker setup. Otherwise, a simple task checklist is sufficient. A beginner need not use an issue tracker for a tiny script.

Copy and paste:

> Read the repository's project rules. Propose the smallest useful feature to build next, with acceptance criteria and a way to test it. Identify which files and data contracts it affects. Implement one reviewable change at a time, run appropriate tests, summarize what changed, and commit only after I approve the result or give you explicit authority to commit.

Result: An incremental development cycle, not one giant untestable generation.

STEP 18 — Let the project tree evolve, but maintain the contracts and instructions

As you build, the agent may add folders, rename modules, change dependencies or propose new architecture. **This is expected.** `scripts/scaffold.sh` records the **initial setup**; it is not an order to reset the project to its original shape. Keep `README.md`, `ARCHITECTURE.md`, `DESIGN.md`, and `DATA_CONTRACTS.md` aligned with the software that actually exists.

Before accepting a change, ask:

> Does this change affect the architecture, UI rules, data producers/consumers, schemas, skills, MCP servers, permissions or build commands? If so, update the relevant documents and tests in the same change. Check schema compatibility and migrations before changing data interfaces. Re-run conflict audits if skills or MCP servers changed. Show me the verification results and any unresolved risks.

Re-run agent initialization or update `AGENTS.md` when the structure changes materially; **review its output because automated initialization can overwrite custom guidance**. Use `improve-`

`codebase-architecture` only after meaningful code exists and you want an architectural review. Commit useful checkpoints and preserve history.

Result: A growing project that remains understandable to both you and future agent sessions.

Appendix — example prompts for maintaining the agent context

When creating or updating `AGENTS.md`, a useful concise instruction is:

> Read `PROJECT_SPEC.md` for scope, `ARCHITECTURE.md` for system invariants, `DESIGN.md` for UI conventions (if present), `DATA_CONTRACTS.md` and `schemas/` for data interfaces (if present), and `SKILLS.md` / `MCP_SERVERS.md` for approved agent capabilities. Read `scripts/scaffold.sh` only when investigating or changing setup. Use the actual build and test commands in this repository; never invent missing files or commit secrets.

A basic project-local skill can be stored as `skills/<skill-name>/SKILL.md` with a clear name, description, when to invoke it, ordered procedure and verification checks. Whether the agent auto-discovers that directory is **tool-specific**; confirm the required path with the agent's documentation. Install sources only after reviewing their content and permissions.

When you change the project tree, **update references**, but do not routinely overwrite manually maintained documents by re-running an agent's initialization command. Review diffs and commit the changes.

Quick completion checklist

- [ ] My first-version goals and boundaries are written in `PROJECT_SPEC.md`.
- [ ]

`ARCHITECTURE.md` states real system rules; `DESIGN.md` exists if a UI is needed.

- [ ] Data producers/consumers are identified; schemas and synthetic tests exist where appropriate.
- [ ] My required skills and MCP servers are distinguished from optional recommendations.
- [ ] Skill/MCP conflicts were reviewed; critical conflicts are resolved before installation.
- [ ] The scaffold creates real files and a runnable starter project.
- [ ] Git has at least one baseline commit; no secrets are committed.
- [ ] My chosen coding agent can read the project rules and invoke approved tools.
- [ ] The baseline tests pass, or failures and blockers are explicitly documented.
- [ ] I understand that the initial tree is a starting point, not a permanent structure.

The entire process in one line

Idea → specification → clarify → design/architecture/data contracts → choose and audit skills/MCP → scaffold → Git baseline → configure tools → initialize agent → verify → build small features → update documents, schemas, tests and Git as the project evolves.

Version control of THIS reusable standard

The guide itself is a **separate living standard**, not the version number of any project you start with it.

- Use `MAJOR.MINOR.PATCH`: **MAJOR** for incompatible major workflow changes; **MINOR** for new compatible phases or guidance; **PATCH** for editorial corrections.
- Every release changes the version in the **title**, **filename** and **metadata**, and adds an entry to the revision history below. Preserve prior versions in Git or a versioned archive. Do not overwrite the old release under its old versioned name.
- Suggested Git commit: `docs(standard): release v1.2.0`; optional tag: `project-standard-v1.2.0`.
- The standard can be used with different operating systems, agents and programming languages. Agent-specific commands may change; check official instructions before executing them.

Revision history

Version	Date	Changes
1.2.0	2026-10-18	Merged Project Bootstrap Kit concepts into one beginner guide: file responsibility matrix, global vs local skills, concrete design tokens, minimum viable configurations, agent-file portability, scaffold regeneration safeguards and explicit human/agent documentation separation.
1.1.0	2026-10-18	Reorganized into a literal beginner workflow; added `PROJECT_SPEC.md`, explicit conditional `DESIGN.md`, `DATA_CONTRACTS.md`, JSON Schema design/verification, clearer tool install timing, real scaffold checks, agent initialization and evolving-project maintenance.
1.0.0	2026-10-18	First versioned baseline: reusable skills, MCP selection and installation, conflict audits, scaffolding, agent context and Git guidance.